RECURRENCIA

PRÁCTICA A2: sesiones 03 y 04

. ***Sesión 03***. *Métodos a evaluar: Exponenciación*

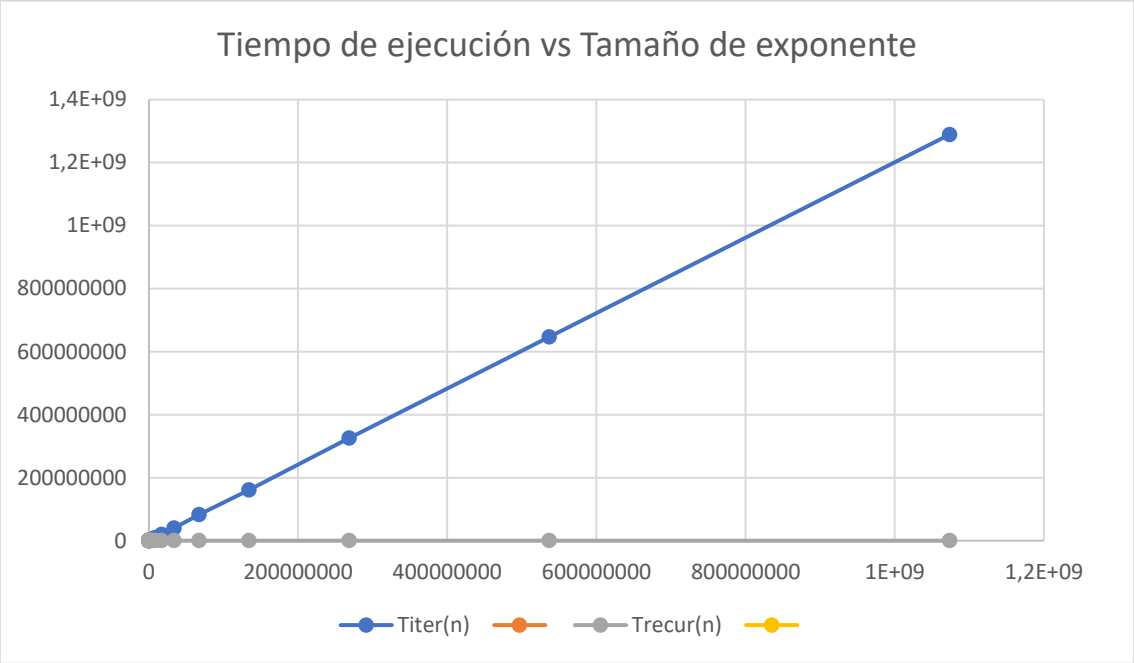
- Una vez hemos implementado en eclipse ambos algoritmos fuerza bruta y divide y vencerás y haber medido mediante la clase TiemposPotencia el tiempo que tardan ambos algoritmos en ejecutarse para distintos tamaños del exponente, pasamos al análisis de datos:

ANÁLISIS DE DATOS

1 – Elaborar una tabla tomando la base = 1, con los tiempos medios obtenidos con los diferentes exponentes, para cada algoritmo. ***He escogido para el exponente valores hasta 1073741824 para que el resultado sea más representativo:***

n	Titer(n)	Trecur(n)
2	122	144
4	122	177
8	144	188
16	255	233
32	388	311
64	777	333
128	1833	311
256	2655	411
512	5266	400
1024	10400	177
2048	15122	211
4096	13222	255
8192	28155	311
16384	60044	222
32768	95011	411
65536	107300	211
131072	181344	222
262144	584711	333
524288	726555	233
1048576	1453055	277
2097152	2608555	355
4194304	5649633	233
8388608	9707955	311
16777216	19774733	300
33554432	40521444	611
67108864	82432800	322
134217728	161422966	344
268435456	325146166	244
536870912	646234966	366
1073741824	1289340611	333

2 – A partir de la tabla anterior, obtener una gráfica en la que representamos los tiempos de los dos algoritmos: tiempo de ejecución *vs* tamaño de exponente.



3 – Obtener de forma teórica el orden de complejidad del algoritmo iterativo.

. Nos encontramos con un bucle for, por lo que sabemos de antemano que será de orden de complejidad O(n). Matemáticamente, si calculamos este orden de complejidad haciendo uso de sumatorios, tenemos:

∑ (desde i=0 hasta n – 1, siendo n = exponente) C = c + n*c

Comprobamos de esta manera que el orden es n.

4 – Consultar la bibliografía indicada en el aula virtual para obtener el orden de complejidad del algoritmo recursivo. Conclusiones sobre la eficiencia de ambos algoritmos, especialmente el algoritmo recursivo.

. Consultando la bibliografía, podemos observar que el algoritmo recursivo tiene un orden de complejidad de O (log n).

Esto supone una mayor eficiencia comparándolo con el del algoritmo iterativo, ya que, el tiempo de ejecución para un método cuyo orden de complejidad es log n es menor que para un orden de complejidad de n.

. ***Sesión 04***. *Métodos a evaluar: Calcular coeficiente binomial*

- Una vez hemos implementado en eclipse ambos algoritmos de Coeficiente Binomial Recursivo (Divide y Vencerás) y Programación Dinámica y haber medido los tiempos mediante la clase TiemposCoeficienteBin, pasamos al análisis de datos:

ANÁLISIS DE DATOS

1 – Elaborar una tabla con los tiempos obtenidos (lo he probado con coeficientes hasta valores de n = 20, ya que el algoritmo tardaba demasiado en ejecutarse a partir de dicho valor):

n	T-recursivo	T-Prog.Din
(8,4)	4100	1311
(10,5)	11111	1888
(12,6)	6933	2288
(14,7)	23277	2811
(16,8)	66311	4200
(18,9)	94122	4322
(20,10)	351155	5422
(22,11)	1295933	6400
(24,12)	5056888	6900
(26,13)	20492600	8088
(28,14)	77269800	10788
(30,15)	291815233	2744
(32,16)	1114873066	4122
(34,17)	4308788444	2944
(36,18)	16747509233	3844
(38,19)	65234582988	4500
(40,20)	255226736688	5144

2 – A partir de la tabla anterior, obtener una gráfica representando los tiempos de ejecución de ambos algoritmos frente a n:



3 – Consultar los apuntes e indicar y justificar cuál es el orden de complejidad de cada algoritmo

. Para el algoritmo recursivo, sabemos de antemano que tiene un orden de O(4^n). Para demostrar esto, cogemos los dos valores más grandes de la tabla, y los sustituimos en la siguiente fórmula:

T(n + 1) / T(n) = T(20) / T(19) = 255226736688 / 65234582988 = 3,91 ≈ 4.

El valor se aproxima bastante, ya que los números que hemos escogido son bastante grandes.

. En cuanto al algoritmo de programación dinámica, el cual en código sería el siguiente:

```java
public int coefBinomialProgDinam() {
    int[][] tabla = new int[N+1][K+1];
    for (int i = 0; i < tabla.length; i++) {
        tabla[i][0] = 1;
    }
    for (int i = 0; i < tabla[0].length; i++) {
        tabla[i][i] = 1;
    }
    for (int i = 2; i < tabla.length; i++) {
        for (int j = 1; j < i && j <= K; j++) {
            tabla[i][j] = tabla[i-1][j-1] + tabla[i-1][j];
        }
    }
    return tabla[N][K];
}
```

Si consideramos que K = n y que N = K * 2, tendríamos que, para los 4 bucles, siendo los dos últimos anidados, el orden del método sería de O(n^2). Comprobemos esto:

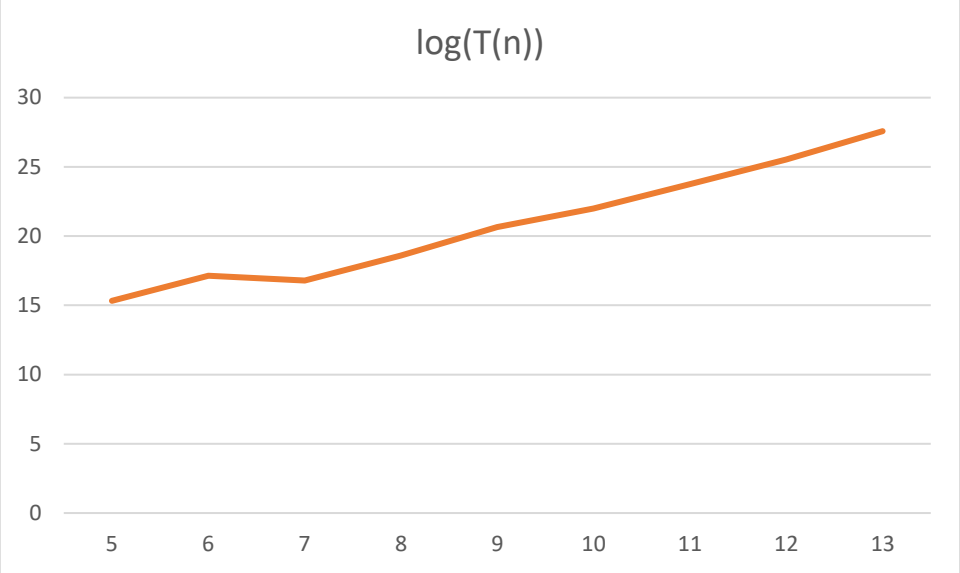
∑ (desde i = 0 hasta 2*n + 1) C + ∑ (desde i = 0 hasta n + 1) C + ∑ (desde i = 2 hasta 2*n + 1) (∑ (desde j = 1 hasta k) D) = 2*n*c + 2*c + n*c + 2*c + d + (2*n^2 + 4*n + n + 2) / 2 = O(n^2).

4 – Demostrar que los resultados experimentales obtenidos para el algoritmo de Programación Dinámica coinciden con los teóricos

. Primero, he elaborado la tabla con logaritmos en base 2:

n	T(n)	log(n)	log(T(n))
32	41077	5	15,3260432
64	144322	6	17,13893171
128	113166	7	16,78808105
256	395355	8	18,59278914
512	1646611	9	20,65106834
1024	4164744	10	21,98979639
2048	14083944	11	23,74754806
4096	48749333	12	25,53887914
8192	200268244	13	27,57735843

. Posteriormente, he realizado la gráfica y he calculado la pendiente de dicha gráfica con Excel:



. La pendiente de dicha gráfica m = 1,52535074. Por lo tanto, experimentalmente, sale un orden de O(n^1,525), el cual no es 2 pero se aproxima también.

5 – Obtener la constante c de la última fila de la tabla de tiempos, e indicar la expresión T(n) para este método con esta implementación:

. Mediante la ecuación de recurrencia, sacamos el valor de c:

T(n) = c * 4^n

255226736688 = c * 4^20

c = 255226736688 / 4^20

c = 0,28606

6 – Calculo del tiempo de ejecución del ambos algoritmos para n = 1024 y k = 512

. Como hemos supuesto antes, para c = 0,28606 y para k = n, aplicamos la ecuación de recurrencia:

Algoritmo recursivo: T(512) = 0,28606 * 4^512 = 8.085 años

Algoritmo dinámico: Primero, hallamos la constante c de la siguiente expresión:

T(n) = c * n^2

T(8192) = c * 8192^2

c = 200268244 / 8192^2 = 3,9546875

Una vez tenemos la constante c, pasamos a calcular el tiempo de ejecución:

T(512) = 3,9546875 * 512^2 = 1022361,625 picosegundos.

7 – Conclusiones sobre la eficiencia de ambos algoritmos.

. Podemos concluir en que el algoritmo recursivo es menos eficiente que el algoritmo dinámico, ya que ambos algoritmos combinan soluciones para subproblemas más pequeños, pero en el caso del algoritmo dinámico el problema que presentan los algoritmos de divide y vencerás de calcular varias veces una solución no lo tiene, ya que los resultados se van almacenando en una matriz.